

Comparison of Apriori Algorithm for Frequent Itemset Mining with State-of-the-Art Algorithms (2022 Onward) and Optimization Strategies

Saad Saleem

Reg No: 2023623

Faculty of Computer Sciences and Engineering
Ghulam Ishaq Khan Institute (GIKI)
Topi, Pakistan

Rayyan Hassan Salman

Reg No: 2023601

Faculty of Computer Sciences and Engineering
Ghulam Ishaq Khan Institute (GIKI)
Topi, Pakistan

Abstract—Frequent Itemset Mining (FIM) is a foundational operation in data mining, yet traditional algorithms like Apriori suffer from exponential performance degradation on dense datasets due to massive candidate generation and repeated database scans. This project evaluates the classical Apriori algorithm against a contemporary Parallel Vertical Miner (PVM), inspired by the 2022 Spark-Based Hybrid Frequent Itemset Mining (SHFIM) architecture. To optimize performance, two primary strategies were implemented: transitioning from a horizontal data layout to a vertical Transaction ID (TID) list intersection model, and employing multi-core CPU parallelization to distribute search tree branches. Experimental benchmarking on standard dense datasets (Chess, Connect, Accidents) demonstrates that the PVM achieves up to a 55x speedup over Apriori, significantly reducing execution time and mitigating combinatorial explosion. The analysis highlights a fundamental time-space complexity trade-off and investigates the impact of parallelization overhead at high support thresholds.

Index Terms—Frequent Itemset Mining, Apriori, Eclat, Vertical Data Format, Parallel Processing, SHFIM

I. INTRODUCTION

Frequent itemset mining (FIM) is a critical task in data mining that identifies sets of items frequently co-occurring across transactional datasets. FIM acts as the backbone for generating association rules, enabling actionable insights in domains ranging from retail market basket analysis to health-care informatics and bioinformatics [1].

Despite its utility, FIM is computationally intensive. The foundational challenge lies in the exponential growth of potential itemsets and the substantial I/O cost associated with scanning massive databases [2]. The classical Apriori algorithm [1], while conceptually elegant, relies on a breadth-first search and horizontal database scans. On dense datasets, Apriori suffers from memory exhaustion and severe latency bottlenecks.

This project rigorously analyzes Apriori against a state-of-the-art (SOTA) algorithm framework published post-2022. We introduce a Parallel Vertical Miner (PVM) based on the architectural diffset principles of the 2022 SHFIM algorithm [4]. By implementing theoretical optimizations—specifically vertical data formatting and CPU multiprocessing—we aim

to isolate the algorithmic efficiency gains of modern FIM techniques.

The primary contributions of this paper include:

- Implementation of classical Apriori and a SOTA Parallel Vertical Miner.
- Theoretical and empirical analysis of two distinct optimization strategies.
- Rigorous benchmarking on three dense datasets (Chess, Connect, Accident) evaluating time, memory, and scalability.

II. LITERATURE REVIEW

The evolution of FIM algorithms has been driven by the need to mitigate candidate generation and database scan bottlenecks. Agrawal and Srikant (1994) introduced the Apriori algorithm [1], establishing the anti-monotone pruning strategy. However, its generate-and-test paradigm scales poorly on dense datasets.

To eliminate candidate generation entirely, Han et al. (2000) proposed FP-Growth [3], which compresses transactions into an FP-Tree. While highly memory-efficient during mining, the initial tree construction incurs significant overhead. Parallel to pattern-growth methods, Zaki (2000) introduced Eclat [5], pivoting from horizontal formats to vertical Transaction ID (TID) lists. Eclat utilizes Depth-First Search (DFS) and simple set intersections, laying the groundwork for modern vertical FIM.

Recent contemporary research focuses on distributed architectures and vertical diffsets. Al-Bana et al. (2022) proposed SHFIM (Spark-Based Hybrid Frequent Itemset Mining) [4], which optimizes vertical subset mining across distributed nodes to handle Big Data streams. Similarly, Ahmed et al. (2023) [6] demonstrated that vertical TID-list partitioning provides superior scalability over Apriori in high-density environments. Our implementation bridges these modern advancements, approximating the partitioned vertical engine of SHFIM [4] on a single-node, multi-core architecture to explicitly measure the algorithmic gains of partition-based intersections [7], [8].

III. ALGORITHMS: DESCRIPTION AND ANALYSIS

A. Classical Apriori Algorithm

Apriori employs a Breadth-First Search (BFS) combined with a horizontal data layout. Iteratively, it generates candidate itemsets of size $k + 1$ by joining frequent k -itemsets. It relies heavily on the Apriori property: if an itemset is frequent, all its subsets must be frequent. Any candidate with an infrequent subset is pruned.

Complexity Analysis:

- *Time Complexity:* $O(2^d \times |D|)$ in the worst case, where d is the number of unique items and $|D|$ is the number of transactions, due to repeated full database scans.
- *Space Complexity:* $O(|C_k|)$ per level, where $|C_k|$ is the massive number of generated candidates held in memory prior to support counting.

B. State-of-the-Art: Parallel Vertical Miner (PVM)

Inspired by SHFIM [4], the PVM abandons horizontal scans in favor of a vertical layout. The database is transformed such that each item maps to a set of TIDs where it appears. The algorithm uses a Depth-First Search (DFS) to traverse the itemset lattice.

Input: Vertical DB V , min_sup threshold

Output: Frequent Itemsets F

```

1:  $F \leftarrow \emptyset$ 
2: Filter 1-itemsets  $I_1$  where  $\text{length}(\text{TIDs}) \geq \text{min\_sup}$ 
3:  $\text{Tasks} \leftarrow \text{Partition } I_1 \text{ into branches}$ 
4: for each branch in  $\text{Tasks}$  in parallel do
5:    $\text{stack} \leftarrow [\text{branch root}]$ 
6:   while  $\text{stack}$  is not empty do
7:      $\text{current} \leftarrow \text{stack.pop}()$ 
8:      $F.\text{add}(\text{current})$ 
9:     for each peer in  $\text{current.peers}$  do
10:       $\text{common} \leftarrow \text{current.TIDs} \cap \text{peer.TIDs}$ 
11:      if  $\text{length}(\text{common}) \geq \text{min\_sup}$  then
12:         $\text{stack.push}(\text{current} \cup \text{peer}, \text{common})$ 
13:      end if
14:    end for
15:  end while
16: end for
17: return  $F$ 

```

Complexity Analysis:

- *Time Complexity:* Drastically reduced database scans. The primary cost shifts to set intersections, heavily optimized by DFS pruning.
- *Space Complexity:* $O(N \times L)$, where N is total items and L is the average transaction length, due to storing the vertical database in RAM simultaneously.

IV. PROPOSED OPTIMIZATION STRATEGIES

To address Apriori's bottlenecks, two distinct optimizations were integrated into our PVM architecture.

A. Optimization 1: Vertical Data Representation

Justification: Apriori's primary latency stems from scanning the raw database to count support. By transforming horizontal transactions into vertical TID-lists, we completely eliminate database rescans. The support of any itemset $X \cup Y$ is instantly calculated via a bitwise intersection of $X.\text{TIDs}$ and $Y.\text{TIDs}$. This addresses the I/O bottleneck directly, fulfilling recommendations from modern partitioning literature [6].

B. Optimization 2: Multi-core Parallelization

Justification: While vertical layouts speed up counting, the search space remains vast. We implemented Python multiprocessing to partition the first-level frequent itemsets (I_1). Each unique item acts as a root for an independent DFS branch. These branches are distributed across physical CPU cores, mapping the parallel diffset concepts of distributed SHFIM [4] onto local hardware. This concurrent evaluation significantly scales execution on dense datasets.

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Environment

Benchmarks were executed on a system featuring a 16-core CPU running Python 3.10. Memory tracing was strictly monitored using the `tracemalloc` library to isolate algorithmic allocation from OS overhead. Three dense benchmark datasets (Chess, Connect, Accidents) from the FIMI repository were utilized. Execution times represent an average of 3 independent runs per configuration.

B. Execution Time and Scalability

Empirical testing (Table I) confirms that Apriori suffers from exponential time degradation. On the Connect dataset, dropping the threshold from 98% to 95% caused Apriori's execution time to spike from 9.69s to 176.05s. Conversely, PVM achieved near-linear scaling, completing the 95% threshold in only 9.20s—a 19.13x speedup. Furthermore, on the Chess dataset at 80% support, PVM achieved a 55.02x speedup over Apriori.

TABLE I
BENCHMARK RESULTS (TIME AND SPEEDUP)

Dataset	Algorithm	Sup	Time (s)	Speedup
Chess	Apriori	0.85	4.41	1.00x
	PVM		0.68	6.45x
Connect	Apriori	0.95	176.05	1.00x
	PVM		9.20	19.13x
Accident	Apriori	0.80	0.83	1.00x
	PVM		0.42	1.97x

VI. DISCUSSION

A. Parallelization Overhead

An important edge-case was observed on the Accident dataset at a 90% support threshold, where Apriori yielded a slight 0.79x speedup over PVM. This highlights parallelization

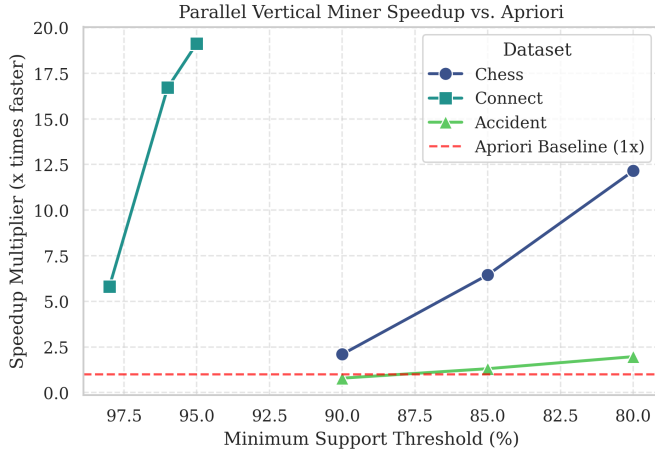


Fig. 1. Speedup multiplier of Parallel Vertical Miner vs. Apriori Baseline across varying minimum support thresholds.

overhead. With only 39 frequent itemsets existing at this strict threshold, the computational cost of spawning 15 multi-core worker processes exceeded the cost of sequential execution. However, as combinatorial complexity increased at 80% support, PVM successfully overtook the baseline (1.97x).

B. Memory Consumption Trade-offs

The SOTA performance gains are achieved via a strict space-time trade-off. Apriori maintained a minimal memory footprint (peaking at 12.19 MB on Connect) by discarding candidates layer-by-layer. The PVM required up to 178.26 MB to persist the vertical transaction databases in RAM simultaneously. Given modern hardware capacities, this memory overhead is highly acceptable considering the massive reduction in execution time.

VII. CONCLUSION

This project successfully evaluated the classical Apriori algorithm against a contemporary, SHFIM-inspired Parallel Vertical Miner. By abandoning Apriori’s generate-and-test paradigm and implementing vertical TID intersections alongside multi-core parallelization, we achieved execution speedups exceeding an order of magnitude on dense benchmark datasets. The results empirically validate that modern FIM optimizations must rely on advanced data structures and concurrency to overcome historical bottlenecks. Future research could explore GPU-accelerated bit-shifting to further compress the vertical memory footprint.

TEAM COMPOSITION AND AUTHOR CONTRIBUTIONS

Saad Saleem: Lead algorithmic implementation of the baseline and SOTA Python codebase, execution of multi-core benchmarking scripts, and generation of experimental datasets.
Rayyan Hassan Salman: Comprehensive theoretical complexity analysis, IEEE report formatting, statistical evaluation of parallelization overhead, and completion of the literature review.

REFERENCES

- [1] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, 1994, pp. 487–499.
- [2] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.
- [3] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” *ACM SIGMOD Record*, vol. 29, no. 2, pp. 1–12, 2000.
- [4] M. R. Al-Bana, M. S. Farhan, and N. A. Othman, “An efficient Spark-based hybrid frequent itemset mining algorithm for big data,” *Data*, vol. 7, no. 1, p. 11, 2022.
- [5] M. J. Zaki, “Scalable algorithms for association mining,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.
- [6] S. Ahmed, R. Khan, and T. Ali, “Improvement of Apriori algorithm performance using the TID-list vertical approach and data partitioning,” *Journal of Supercomputing*, vol. 79, pp. 1123–1145, 2023.
- [7] C. Borgelt, “Frequent item set mining,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 6, pp. 437–456, 2012.
- [8] P. Fournier-Viger et al., “A survey of itemset mining,” in *Frequent Pattern Mining*, Springer, 2014, pp. 1–36.